

PROJECT SUBMISSION

Paper Forge

AI-powered A-Level exam paper generation for teachers — upload your papers, get a new one in 90 seconds.

TEAM	TRACK	BUILD TIME	ENGINEERS
ShipHappens	Gemini	20 Hours	4 Developers

CONFIDENTIAL
· FOR
AUTHORIZED
USE ONLY

CURSOR

GEMINI

SUPABASE

VERCEL

ELEVENLABS

OPENAI

MIRO

TABLE OF CONTENTS

Project Submission Document

SECTION 01

Project Overview

SECTION 02

Problem Statement

SECTION 03

Proposed Solution

SECTION 04

Functional Requirements

SECTION 05

Non-Functional Requirements

SECTION 06

Technical Architecture

SECTION 07

Security

SECTION 08

User Stories & Use Cases

SECTION 09

Target Users & Market

SECTION 10

Business Model

SECTION 11

Why This Project Leads the Track

SECTION 12

Team & Roles

SECTION 01

Project Overview

— PROJECT NAME & ONE-LINE PITCH

PaperForge — an AI-powered exam paper generator that turns 3–5 uploaded A-Level English papers into a brand-new, constraint-matched exam paper in under 90 seconds, giving teachers back hours of drafting time every week.

— SUMMARY

A-Level English teachers spend 4–8 hours manually drafting each practice paper — cross-checking mark allocations, question variety, difficulty balance, and syllabus alignment, all under the time pressure of exam season. PaperForge solves this by extracting the structural DNA of a set of uploaded papers into a precise "blueprint" (sections, marks, question types), then instructing Google Gemini to generate a fresh paper that satisfies every constraint exactly. The output appears live in the browser via streaming, is fully editable at the question level, pairs with an automatically generated answer key and marking scheme, and exports to a print-ready PDF. The primary users are A-Level and O-Level English teachers at tutoring centres and secondary schools, particularly across the UK and Southeast Asia where Cambridge qualifications dominate.

— SUBMISSION DETAILS

Track: Gemini

Track technology integration: Google Gemini is the core engine. Three distinct Gemini capabilities are used: `gemini-2.5-pro` / `gemini-2.5-flash` for structured paper generation and blueprint extraction (Gemini Files API + `response_schema`), `gemini-2.5-flash` for per-question regeneration and answer key generation, and `text-embedding-004` for embedding-based duplicate content detection.

Team name: ShipHappens

Demo URL: TBA (deployed on Vercel + Railway)

Repository: TBA (GitHub — ShipHappens/PaperForge)

SECTION 02

Problem Statement

— THE PROBLEM

A-Level and O-Level English teachers at tutoring centres and secondary schools spend **4–8 hours drafting a single practice exam paper**. This is not a one-off effort — teachers need a continuous supply of fresh papers, typically 1–4 new papers per exam season per subject, to ensure students are not repeating questions they have already seen. Each paper must simultaneously maintain correct total marks, balance question types across sections (MCQ, structured response, essays), match the appropriate reading level and passage length, and align to the Cambridge Assessment International Education syllabus.

If left unsolved, teachers either produce fewer practice papers — directly harming student outcomes at a high-stakes stage — or burn out from the drafting workload on top of their teaching, marking, and administrative duties.

— WHY IT MATTERS

The problem peaks twice a year: April–May and October–November, when A-Level exam seasons align. During these windows, the demand for practice materials spikes exactly when teachers have the least time to produce them.

Current workarounds all fall short:

- **Hand-editing old papers** — slow, risks students recognising questions, and requires full re-typesetting for changed mark allocations.
- **Third-party question banks** (e.g., Hodder Education, licensed Cambridge packs) — expensive (£200–500/year per teacher), inflexible, and not customisable to a school's specific teaching style or local cultural context.
- **Generic AI tools** (ChatGPT, Gemini web) — require the teacher to be skilled at prompt engineering, produce unstructured output, perform no marks accounting, and cannot export a formatted paper.

— ROOT CAUSE

There is no tool that understands the **structural DNA of an exam paper** — the precise distribution of marks, question types, instruction styles, reading passage requirements, and section constraints

— and can reproduce that structure with entirely fresh content at machine speed. The problem is not that AI cannot write exam questions. The problem is that nobody has built the pipeline that extracts constraints from real papers, enforces them during generation, validates the output, and delivers it in a form teachers can actually use.

SECTION 03

Proposed Solution

— WHAT THE PRODUCT DOES

PaperForge is a browser-based tool with a four-step wizard: **Upload** → **Blueprint Review** → **Editable Paper** → **Export**.

The teacher drops 3–5 existing A-Level papers onto the upload page. Each PDF is stored in Supabase Storage and uploaded to the Gemini Files API, where it is cached for 48 hours. The teacher clicks "Extract Blueprint" — Gemini analyses all papers simultaneously and returns a `BlueprintSchema`: a structured JSON object encoding the exact section layout, marks per section, marks per question, question types (MCQ, structured, essay), context-passage requirements, and typical instruction phrasing. The teacher reviews the blueprint and can adjust any field before proceeding.

Clicking "Generate Paper" opens a Server-Sent Events stream. Gemini generates a full paper constrained by the blueprint — the backend post-validates that marks totals are within 10% of the target and retries if not. Section completion events fire in real time so the teacher sees the paper building out live rather than staring at a spinner. The result is a fully editable paper in the browser. The teacher can edit any question inline, regenerate a single question with an optional nudge instruction, or regenerate an entire section. A parallel Gemini call generates a complete answer key and marking scheme on demand. A duplicate-detection check (embedding cosine similarity against the source PDFs) flags any generated question that is too close to source content. When satisfied, the teacher exports to a print-ready PDF.

— KEY FEATURES

FEATURE	USER NEED ADDRESSED
PDF upload + Gemini Files API caching	No re-upload cost; up to 5 source papers analysed together
Blueprint extraction	Teacher sees and can correct the structural spec before generation
Constrained paper generation	Marks totals and question types match the source papers exactly

SSE streaming generation	Real-time progress — paper appears section by section, not all-at-once
Inline editing + per-question regeneration	Teachers make small corrections without restarting the whole flow
Per-section regeneration	Replace a whole section that didn't land without touching the rest
Answer key + marking scheme	Produces hand-out-ready marking criteria, section by section
Difficulty scoring (1–5) per question	Balanced difficulty distribution visible before printing
Duplicate content guard	Warns if a generated question is too similar to a source paper
PDF export	Print-ready output; no reformatting needed

— SCOPE

In scope:

- All 10 features above, built and demonstrable
- A-Level English (Cambridge syllabus)
- Anonymous session model (no login required)
- Cost and token tracking per AI call

Deliberately out of scope:

- User authentication and persistent accounts
- Multi-subject support (Maths, Sciences)
- LMS integrations (Google Classroom, Canvas)
- Collaborative editing by multiple teachers
- Server-side headless PDF rendering (uses browser print)

SECTION 04

Functional Requirements

MUST HAVE

- The system shall accept 3–5 PDF uploads per session and store them in Supabase Storage.
- The system shall upload each PDF to the Gemini Files API and cache the file URI to avoid re-upload on retry.
- The system shall extract a `BlueprintSchema` — sections, marks totals, question types, instruction phrasing — from the uploaded PDFs via a single Gemini call.
- The system shall generate a full exam paper constrained by the blueprint, with marks drift $\leq 10\%$ from the target total.
- The system shall stream generation progress to the frontend via Server-Sent Events, emitting section-completion events in real time.
- The system shall display the generated paper with inline text editing on any question field.
- The system shall support per-question regeneration with an optional teacher nudge string.
- The system shall export the current paper state to a print-ready HTML/PDF via a Jinja2 template.

SHOULD HAVE

- The system should generate a complete answer key and marking scheme for any paper on demand.
- The system should assign a difficulty score (1–5) to every question during generation.
- The system should check generated questions against source PDFs for duplicate content using embedding cosine similarity.
- The system should support per-section regeneration (replacing all questions in one section).
- The system should fall back from `gemini-2.5-pro` to `gemini-2.5-flash` automatically on quota exhaustion.
- The system should record token usage and estimated cost (USD) for every Gemini call in `api_usage_log`.

COULD HAVE / WON'T HAVE (THIS BUILD)

- User authentication and persistent paper library (out of scope — anonymous sessions only).
 - Multi-subject support beyond A-Level English (out of scope — prompt re-engineering required).
 - LMS export to Google Classroom or Canvas (post-launch).
 - Collaborative editing (multiple teachers on one paper simultaneously).
 - Cost/token dashboard in the UI (logged to DB; UI display is post-MVP).
 - Server-side headless PDF renderer (browser print dialog is sufficient for MVP).
-

SECTION 05

Non-Functional Requirements

— PERFORMANCE

- Blueprint extraction from 5 PDFs: **≤15 seconds**
- Paper generation (first SSE event): **≤5 seconds**; complete paper: **≤90 seconds**
- Per-question regeneration: **≤10 seconds**
- Answer key generation: **≤20 seconds**
- PDF export render: **≤2 seconds**

— RELIABILITY & ERROR HANDLING

- All Gemini calls wrapped in a retry loop (max 3 attempts, exponential back-off)
- Quota exhaustion (HTTP 429): automatic model fallback from `gemini-2.5-pro` → `gemini-2.5-flash`
- Post-validation on every AI response; marks drift >10% triggers a retry rather than returning bad output
- Corrupted or too-short PDFs rejected at upload with a clear user-facing message
- Mid-stream Gemini errors emit a single `{event:"error"}` SSE message and close the connection cleanly

— USABILITY

- Wizard-style navigation with clear step indicators
- Mobile-responsive layout (shadcn/ui + Tailwind CSS)
- SSE progress events eliminate the blank-spinner problem during long generation
- Inline editing requires no modal or separate save step
- Accessible colour contrast (WCAG AA) in the default theme

— SCALABILITY

- FastAPI backend is stateless; session state lives in Supabase (scales horizontally)
- Gemini Files API 48-hour cache eliminates repeat-upload token cost as teacher iterates
- `pgvector` extension installed and `question_embedding` table ready for a cross-session question bank in v1 without schema migration
- Dedup guard runs in-process per-request in MVP; migrates to a persistent vector index in v1
- Supabase handles connection pooling; Railway/Fly.io backend scales by adding replicas

SECTION 06

Technical Architecture

— SYSTEM OVERVIEW



```
| text-embedding-004 | | generated_papers · api_usage |
| Files API (48h cache) | | Storage: source-pdfs bucket |
└──────────────────┘ └────────────────────────────────┘
```

— TECHNOLOGY STACK

LAYER	TECHNOLOGY	RATIONALE
Frontend	Next.js 14 App Router, TypeScript, Tailwind CSS, shadcn/ui, Zustand, SWR, react-dropzone	Production-grade React with type safety; shadcn/ui eliminates custom component work for junior FE
Backend	FastAPI, Uvicorn, SQLModel, Pydantic, Alembic, python-multipart	Async-first Python; Pydantic schema-sharing between API layer and AI output validation
AI — generation	google-genai SDK, <code>gemini-2.5-pro</code> → <code>gemini-2.5-flash</code> fallback	Best-in-class multimodal reasoning; structured JSON output via <code>response_schema</code> eliminates prompt parsing
AI — embeddings	<code>text-embedding-004</code> (Gemini)	Same API key; L2-normalised output makes cosine similarity a plain dot product
Database	Supabase managed PostgreSQL + pgvector	Zero-ops Postgres; pgvector ready for v1 question bank; Row-Level Security for future auth
File storage	Supabase Storage	Consistent with DB vendor; private buckets with signed URL access
PDF processing	PyMuPDF (<code>fitz</code>)	Fast C-extension; used for text extraction in dedup and metadata fallback
PDF export	Jinja2 + print CSS	Zero browser-side dependencies; teacher prints from native browser dialog
Hosting	Vercel (frontend), Railway/Fly.io (backend)	Zero-config deploys; both have free tiers sufficient for hackathon demo

— DATA FLOW

- 1

Upload: Teacher drops 3–5 PDFs → Next.js → `POST /sessions/{id}/files` → FastAPI saves to Supabase Storage → uploads each to Gemini Files API → caches `gemini_file_uri` in `source_files` table.
- 2

Blueprint: `POST /sessions/{id}/blueprint` → FastAPI calls `extract_blueprint(file_uris)` → Gemini returns `BlueprintSchema` JSON → stored in `sessions.blueprint`.

- 3 **Generate (streaming):** `POST /sessions/{id}/generate/stream` → `generate_paper_stream()` calls `generate_content_stream` → SSE events: `start` → `chunk` × N → `section` × sections → `complete {paper}`.
- 4 **Edit / Regenerate:** `PATCH /sessions/{id}/paper` for inline edits; `POST /sessions/{id}/regenerate/question` for Gemini-powered question swap; `POST /sessions/{id}/regenerate/section` for full-section replacement.
- 5 **Answer Key:** `POST /sessions/{id}/answer-key` → `generate_answer_key(paper)` → `AnswerKeySchema` returned and displayed alongside the paper.
- 6 **Dedup:** `POST /sessions/{id}/dedup` → `check_duplicates(paper, source_pdfs)` → embeddings compared via cosine similarity → `DuplicateWarning[]` with severity returned to UI.
- 7 **Export:** `GET /sessions/{id}/export` → FastAPI renders Jinja2 HTML template → returned to Next.js → browser print dialog → PDF.

— AI INTEGRATION

Provider: Google Gemini via the `google-genai` Python SDK.

Models: `gemini-2.5-pro` for initial paper generation (most demanding reasoning task); `gemini-2.5-flash` for blueprint extraction, question regeneration, and answer key (speed-critical, lower cost); `text-embedding-004` for duplicate-content embeddings.

Prompting strategy: Every AI call uses `response_schema=<PydanticModel>` via `response_mime_type="application/json"` — Gemini returns machine-validated JSON, not free text. Each module (`blueprint.py`, `generate.py`, `regenerate.py`, `answer_key.py`) maintains its own prompt template encoding explicit constraints: marks totals, question counts, marks-per-sub-part sums, no verbatim copying of source passages, difficulty distribution guidelines, and cultural relevance nudges.

Shared wrapper: `ai/wrapper.py:call_structured()` handles retries (max 3), post-validation callbacks, model fallback, and per-call `UsageRecord` logging. No AI module calls the SDK directly; all calls go through the wrapper.

— KNOWN TECHNICAL LIMITATIONS

- Anonymous sessions only — session UUIDs provide no cryptographic isolation between users.
- `gemini-2.5-pro` free-tier quota is 0 RPM; a paid key is required for the pro-model generation path.
- Dedup embedding check runs per-request with no persistent index; large source corpora will slow this call.
- Export uses the browser print dialog; a server-side headless PDF renderer is needed for bulk export.

- Subject scope is A-Level English only; other subjects require prompt re-engineering.
-

SECTION 07

Security

— AUTHENTICATION & AUTHORISATION

PaperForge uses anonymous sessions in this build — no login is required. A UUID4 session ID is generated on the user's first visit and stored in `localStorage`. All backend resources (source files, blueprints, generated papers) are scoped to the `session_id` column in every database table. Supabase Row-Level Security (RLS) policies are defined and ready; they are not enforced in MVP because there is no JWT to verify against, but the column structure means enabling auth is a one-line schema change for v1.

— DATA HANDLING

Uploaded PDFs are stored in a private Supabase Storage bucket; they are accessible only via signed URLs generated server-side. No PII is collected — teachers are fully anonymous. Session data (blueprints, generated papers) lives in Supabase PostgreSQL scoped to `session_id`. Gemini Files API stores uploaded PDFs for 48 hours and then auto-deletes them — no long-term storage on Google's side.

— API & SECRET MANAGEMENT

All secrets — `GOOGLE_API_KEY`, Supabase service key, `DATABASE_URL` — are stored

— INPUT VALIDATION

- **File type:** Only `application/pdf` MIME type accepted at upload; non-PDF files are rejected with HTTP 422 before they touch storage.
- **File size:** Configurable maximum (default 20 MB) enforced by FastAPI's request body limit.
- **AI output:** All Gemini responses are validated against Pydantic schemas (`BlueprintSchema`, `PaperSchema`, `AnswerKeySchema`); malformed output triggers a retry rather than passing through to the UI.
- **SQL:** All database access goes through SQLAlchemy's ORM layer — no raw string interpolation; SQL injection is not possible via the application layer.
- **Session IDs:** FastAPI validates that the `session_id` path parameter is a valid UUID4 before querying the database.

— KNOWN VULNERABILITIES OR GAPS

- Session isolation is UUID-only — if a session ID is shared or guessed, another user can read that session's papers. Accepted for MVP hackathon use; fixed in v1 with JWT auth.

exclusively in `backend/.env`, which is listed in `.gitignore` and has never been committed to the repository. The frontend uses only the Supabase publishable anon key (safe for client-side exposure by design). No secrets appear in the Next.js bundle or any environment variable prefixed `NEXT_PUBLIC_` except the publishable key.

- No rate limiting on API endpoints — acceptable for a controlled demo; required before public launch.
- Gemini-generated content is not moderated for harmful material — acceptable for an internal school tool with trusted users.

SECTION 08

User Stories & Use Cases

— CORE USER STORIES

TEACHER

As a teacher, I want to upload my existing A-Level English papers so that PaperForge understands the exact exam structure and mark allocations I use.

TEACHER

As a teacher, I want to review the extracted blueprint before generation so that I can catch any misinterpretations of my paper's structure before they propagate into the new paper.

TEACHER

As a teacher, I want a new exam paper generated in under 2 minutes so that I can produce fresh practice papers without blocking my evening on drafting.

TEACHER

As a teacher, I want to edit any question's wording inline so that I can make small adjustments without restarting the entire generation flow.

TEACHER

As a teacher, I want to regenerate a single question with a nudge ("make this harder", "use a Sri Lankan context passage") so that I can improve individual questions without redoing the whole paper.

TEACHER

As a teacher, I want an automatically generated answer key with mark-by-mark criteria so that I can hand it to a teaching assistant for marking without writing it myself.

TEACHER

As a teacher, I want a warning if any generated question is suspiciously similar to a question from my source papers so that my students are not practicing on questions they have already seen.

TEACHER

As a teacher, I want to export the final paper as a PDF so that I can print it and distribute it in class without further formatting work.

— PRIMARY USE CASE WALKTHROUGH

- 1 Teacher navigates to PaperForge. A session UUID is generated silently; the teacher sees the Upload page.
- 2 Teacher drags 3 PDF papers onto the upload zone. Files upload to Supabase Storage; FastAPI uploads each to the Gemini Files API and caches the file URLs.
- 3 Teacher clicks "Extract Blueprint". Gemini analyses all 3 papers simultaneously and returns a `BlueprintSchema` — sections, marks, question types, instruction phrasing. Blueprint appears on screen within 10 seconds.
- 4 Teacher reviews the blueprint, sees it correctly identified 3 sections (MCQ, structured response, essay), confirms, and clicks "Generate Paper".
- 5 A streaming connection opens. The UI shows "Generating Section A...", "Section A done — Section B in progress...". The complete paper appears within 60–90 seconds.
- 6 Teacher reads through the paper, tweaks a question's wording inline, and regenerates one essay question with the nudge "use a modern Sri Lankan context". New question appears in ~8 seconds.
- 7 Teacher clicks "Check for Duplicates". Dedup guard returns one medium-severity warning on a structured-response question. Teacher regenerates that question.
- 8 Teacher clicks "Generate Answer Key". Marking scheme appears alongside the paper in ~12 seconds.
- 9 Teacher clicks "Export PDF". Browser print dialog opens with a cleanly formatted paper. Teacher saves as PDF and sends to the photocopier.

— EDGE CASES

- **Corrupted PDF:** FastAPI catches the PyMuPDF parse error and returns HTTP 422 with a message: "Could not read this PDF. Please check the file is not password-protected or corrupted."
- **Too-small PDF** (<2 pages, <500 words): Blueprint extractor raises a post-validation error and tells the user the paper is too short to analyse reliably.
- **Marks drift >10%:** `generate.py` post-validator retries the Gemini call; after 3 retries, returns the best-effort result with a visible warning in the UI.

- **Quota exhaustion (429):** `wrapper.py` catches the error and retries with `gemini-2.5-flash` automatically; the teacher sees no interruption.
 - **Mid-stream disconnect:** SSE error event fires, UI shows "Generation interrupted — retry?" with a button that resumes from the last confirmed section.
-

SECTION 09

Target Users & Market

— PRIMARY USER

A-Level and O-Level English teachers at private tutoring centres and independent secondary schools. Specifically: teachers who are responsible for producing 1–4 new practice papers per exam season per subject, and who currently do this entirely manually. The pain is worst in the 6-week windows before the May/June and October/November Cambridge examination sittings, when demand for practice materials spikes exactly as teacher bandwidth shrinks due to revision classes, mock exams, and marking loads.

Secondary user: **exam coordinators** at schools with multiple teachers delivering the same subject — they need a consistent paper format across teachers and would use PaperForge as a shared paper-generation service rather than having each teacher produce their own.

— MARKET OPPORTUNITY

SEGMENT	SIZE	ENTRY WEDGE
UK private tutoring centres (Cambridge A-Level)	~5,000 centres, est. 15,000 teachers	Direct outreach; team has UK academic contacts
Sri Lanka Cambridge A/O Level schools	~800 schools, ~4,000 relevant teachers	Team is Sri Lankan; direct tutoring centre connections
Singapore / Malaysia Cambridge cohort	~2,000 schools, est. 10,000 teachers	Phase 2 expansion via teacher Facebook communities
Global Cambridge cohort (130+ countries)	~10,000 schools	Long-term via partner channel and Cambridge licensing

Total addressable market at £12/teacher/month: **~£18M ARR** for the UK + Sri Lanka + Singapore segment alone. Initial serviceable market (independent tutoring centres, early adopters): **~£2M ARR**.

— COMPETITIVE LANDSCAPE

COMPETITOR	WHAT THEY OFFER	KEY WEAKNESS
Cambridge-licensed question banks (Hodder Education, PastPapers.co)	High-quality curated questions, syllabus-aligned	£200–500/year per teacher; zero customisation; same questions available to all students; no paper assembly
Generic AI tools (ChatGPT, Gemini web UI, Claude)	Can draft individual questions on request	No marks accounting; unstructured output; teacher must format, assemble, and validate the full paper manually; no PDF export; no answer key
Manual drafting (current default for most teachers)	Full editorial control; perfectly customised	4–8 hours per paper; not scalable; relies on individual teacher's time and expertise; no duplication detection

PaperForge's differentiation: The only tool that (a) ingests real exam papers as the specification source, (b) enforces marks constraints automatically, (c) produces structured, editable output with an answer key, and (d) checks its own output for duplicate content — all in one flow, in under 2 minutes.

SECTION 10

Business Model

— REVENUE MODEL

SaaS subscription with a freemium entry tier:

TIER	PRICE	WHAT'S INCLUDED
Free	£0	2 paper generations / month, no answer key, no dedup guard, watermarked PDF export
Pro	£12 / teacher / month	Unlimited generations, answer key + marking scheme, dedup guard, clean PDF export, priority generation queue
School	£99 / school / month	Up to 20 teachers, shared paper library, cross-teacher duplicate detection, export to shared Google Drive

Unit economics: Gemini cost per full paper generation (5 PDFs + blueprint + paper + answer key) ≈ \$0.08–0.14 at flash-model rates, well below the implied £0.40/generation at Pro pricing even at 30 generations/month.

— PRICING HYPOTHESIS

£12/month for a Pro teacher subscription saves a minimum of 4 hours of drafting per paper generated. At a conservative teacher hourly value of £25/hr, a single paper saves £100 of time — making the subscription pay for itself in the first use of the month. Teachers already pay £200–500/year for static question banks that require manual paper assembly; PaperForge delivers a finished paper and undercuts that price while doing more work.

— GO-TO-MARKET

- MONTH 0-1

Direct outreach to 10 tutoring centre owners the team knows personally in Sri Lanka. Offer 90-day Pro free trial in exchange for a structured case study (time-saved measurements, teacher feedback). Goal: 3 paying centres by month 2.
- MONTH 2-3

Expand via teacher Facebook groups (A-Level Teachers Sri Lanka, UK Cambridge Teachers Network). Referral programme: Pro teachers who refer a

colleague get one free month. Target: 50 Pro subscribers.

MONTH 4-6

Approach school exam coordinators with the School tier. One School contract (£99/month, 20 teachers) replaces 20 individual Pro slots — lower CAC, higher retention. Target: 5 School contracts.

— ROADMAP

NOW (THIS BUILD)

Upload → Blueprint → Generate (streaming) → Edit → Regenerate (question + section) → Answer Key → Dedup Guard → Export PDF. All functional. A-Level English only.

0-3 MONTHS

User authentication + persistent paper library. Multi-subject support (Maths, Sciences — prompt re-engineering). Cross-session question bank using pgvector (already instrumented). Miro export for visual lesson planning boards.

12 MONTHS

Full Cambridge IGCSE + A-Level syllabus coverage. LMS integrations (Google Classroom, Canvas). Collaborative editing (multiple teachers on one paper). AI difficulty calibration against past grade boundaries. Mobile app for on-the-go question approval.

SECTION 11

Why This Project Leads the Track

— TECHNICAL EDGE

PaperForge uses **three distinct Gemini capabilities** within a single coherent product: `gemini-2.5-pro/flash` for structured multimodal reasoning (blueprint extraction from PDFs, paper generation), `gemini-2.5-flash` for speed-critical sub-tasks (question regeneration, answer key), and `text-embedding-004` for semantic duplicate detection. This is not a wrapper around a single Gemini chat call — it is a multi-model AI pipeline where each model is chosen for the right job.

- **Structured JSON at every AI boundary** — every Gemini call uses `response_schema=<PydanticModel>`. No prompt parsing, no regex, no "hope the model formats it correctly". The output is machine-validated JSON or the call retries.
- **Production-grade SSE streaming** — real-time paper generation via Python async generators through FastAPI's `StreamingResponse` to a Next.js `EventSource`. Section completion events fire as each section of the paper is completed, not as raw token chunks.
- **Self-rejecting AI pipeline** — the backend post-validates every generated paper against marks constraints. If the total drifts more than 10%, it rejects its own output and retries. The system is opinionated about quality, not just speed.
- **pgvector-ready architecture** — the duplicate guard uses `text-embedding-004` + cosine similarity today and will migrate to a persistent cross-session vector index with zero schema changes in v1.

— PROBLEM-SOLUTION FIT

The problem is **real, specific, named, and quantified**: A-Level teachers spend 4–8 hours per paper. Every feature in PaperForge traces directly to a step in that drafting process. Blueprint extraction replaces the manual structure-counting step. Constrained generation replaces the blank-page drafting step. Inline editing and per-question regeneration replace the revision step. Answer key generation replaces the marking-scheme-writing step. Duplicate detection replaces the manual

cross-check step. The product does not add features for demo purposes — every feature exists because a teacher needed it.

— EXECUTION QUALITY

In 20 hours, the team shipped: a full-stack web application with a production FastAPI backend, a Next.js frontend with a multi-step wizard, 7 distinct REST and SSE endpoints, a shared AI wrapper with retry and fallback logic, 5 separate Gemini-integrated AI modules, Supabase PostgreSQL + Storage integration, and a Jinja2 PDF export pipeline. The entire AI layer is tested with sample scripts that can be run in isolation — the demo does not depend on live Gemini calls to look good.

— REAL-WORLD POTENTIAL

This does not end at the hackathon. Three tutoring centres have already expressed interest in piloting. The architecture is designed for the next phase: pgvector is instrumented, Supabase RLS is ready for auth, and the prompt templates are parameterised by subject so adding Maths or Sciences is a configuration change, not a rewrite. The Cambridge A-Level cohort spans 130+ countries — the addressable market is genuinely large, and the team has direct distribution into the initial beachhead segment.

SECTION 12

Team & Roles

AI ENGINEER**AI Engineer — ShipHappens**

Built: The entire `backend/ai/` module — Gemini SDK client, shared retry wrapper (`wrapper.py`), blueprint extraction (`blueprint.py`), paper generation with streaming (`generate.py`), per-question and per-section regeneration (`regenerate.py`), answer key generation (`answer_key.py`), embedding-based duplicate guard (`dedup.py`), all Pydantic schemas (`schemas.py`), and sample-generation scripts. Led all prompt engineering, post-validation logic, and retry strategy. Proved out the Gemini SDK pattern in the first 2 hours of the build — establishing the foundation all three other engineers built on top of.

FULL-STACK ENGINEER 1 — BACKEND LEAD**Full-Stack 1 — ShipHappens**

Built: FastAPI application scaffolding (`backend/app/main.py`), all REST and SSE API endpoints (`app/routers/` — blueprint, generate/stream, regenerate, answer-key, dedup, export, files), session management, Supabase client wiring, and endpoint integration tests. Responsible for the API contract that the frontend consumed from h2 onward.

FULL-STACK ENGINEER 2 — DATA LEAD**Full-Stack 2 — ShipHappens**

Built: Supabase project setup, Storage bucket configuration, SQLAlchemy database models (`app/models/`), Alembic migration files, Supabase Storage adapter service, seed data and smoke tests, and the Jinja2 HTML export template with print CSS. Owns the data layer end-to-end — from schema definition to the final PDF export.

FRONTEND ENGINEER**Frontend — ShipHappens**

Built: Next.js 14 App Router scaffolding, the four-step wizard UI (Upload page, Blueprint Review page, Editable Paper page with inline editing and per-question regeneration, Export Preview page), Zustand state management, SWR data fetching, `EventSource` SSE consumer for streaming generation, and all loading/empty/error states. Responsible for every pixel the teacher sees.

— WHY THIS TEAM

The team covers every layer of the stack with zero overlap and zero gap: AI pipeline, backend API, data and persistence, and frontend UI. Each engineer owned a vertical slice that could be developed in parallel against a shared JSON contract (the Pydantic schemas). The AI engineer locked the API pattern in the first 2 hours — giving the backend and frontend engineers a stable foundation to build on immediately. Junior-level execution was mitigated by strong task decomposition, a shared retry and validation framework, and a sprint structure with hard milestones every 6 hours. The team ships.

CURSOR BUILDATHON · SHIPHAPPENS · PAPERFORGE · CONFIDENTIAL
